形式语言与自动机

Part 3 正则表达式

主讲人： 谭家骐

2023.8.16

内容大纲

- **正则表达式**
  - 正则表达式的递归定义
  - 正则表达式的**代数定律**
  - 正则表达式示例
- **有穷自动机和正则表达式**

正则表达式

- **有穷自动机**
  - 通过抽象机器装置扫描和接受字符串识别（表示）正则语言
- **正则表达式**
  - 通过代数式表示或产生正则语言，便于正则语言的计算机处理
  - 正则表达式所表示的语言与正则语言等价

语言运算

- **设L和M是两个语言，那么**

  并： $L \cup M = \{w | w \in L$或$w \in M\}$

  连接： $L \cdot M = \{w | w = xy, x \in L$且$y \in M\}$

  幂： $L^0 = \{\epsilon\}$，$L^1 = L$，$L^n = L^{n-1} \cdot L$，

  克林闭包： $L^* = \bigcup_{i=0}^{\infty} L^i$

例子

- 例:若有$L = \{0,11\}, M = \{\varepsilon, 001\}$，那么
  - $L \cup M = \{0,11, \varepsilon\ , 001\}$
  - $L \cdot M = \{0,0001,11,11001\}$
  - $M \cdot L = \{0,11,0010,00111\}$
  - $L^0 = \{\epsilon\}, L^1 = L, L^2 = \{00,011,110,1111\}$

- 例:对于空语言$\emptyset$
  - $\emptyset^0 = \{\varepsilon\}, \emptyset^n = \emptyset, \emptyset^* = \{\varepsilon\}$

正则表达式的递归定义

- 基础1：$\emptyset$是一个正则表达式，表示空语言$\emptyset$。
- 基础2：ε是一个正则表达式，表示语言$\{\varepsilon\}$。
- 基础3：对于任意一个符号a，a是一个正则表达式，表示语言$\{a\}$，其有一个长度为1的字符串。

正则表达式的递归定义

- **归纳1：**如果E_1和E_2是正则表达式，那么$E_1 + E_2$也是正则表达式，且$\mathrm{L}(E_1 + E_2) = \mathrm{L}(E_1) \cup \mathrm{L}(E_2)$。
- **归纳2：**如果E_1和E_2是正则表达式，那么E_1E_2也是正则表达式，且$\mathrm{L}(E_1E_2) = \mathrm{L}(E_1)\mathrm{L}(E_2)$。
- **归纳3：**如果E是正则表达式，则E^*也是正则表达式。
- **归纳4：**如果E是正则表达式，则(E)也是正则表达式，表示语言$\mathrm{L}(E)$。

运算符的优先级

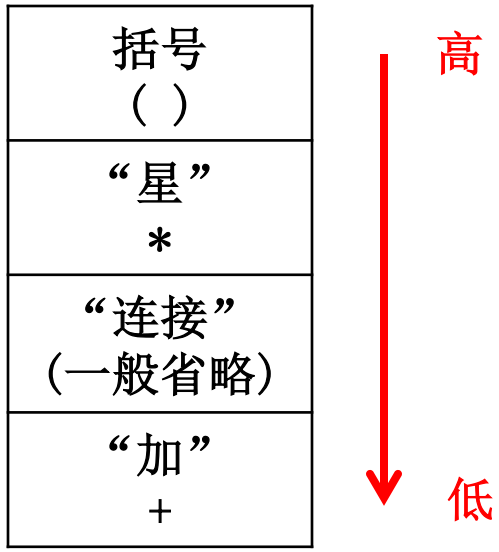
- **正则表达式中三种运算以及括号的优先级：**

(1) “括号”优先级最高

(2) “星”运算: E^*

(3) “连接”运算:E_1E_2

(4) “加”最低: $E_1 + E_2$



正则表达式示例

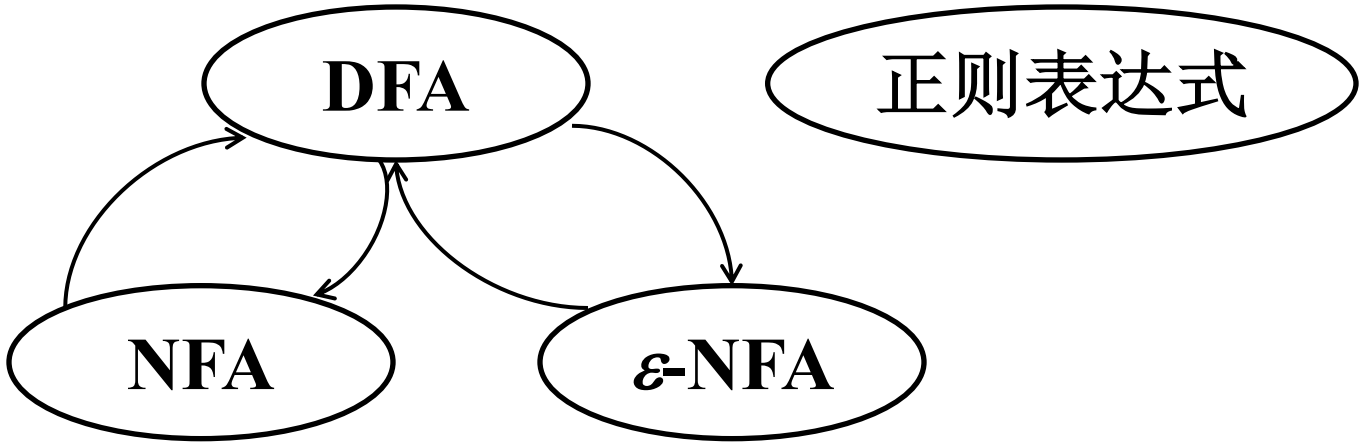
- **给出正则表达式$(a+b)^*(a+bb)$定义的语言**
  - $a \rightarrow \{a\}$，$b \rightarrow \{b\}$
  - $a + b = \{a\} \cup \{b\} = \{a, b\}$，$bb \rightarrow \{b\}\{b\} = \{bb\}$
  - $(a + b)^* = \{a, b\}^*$
  - $(a + bb) = \{a, bb\}$
  - $(a + b)^*(a + bb) = \{a, b\}^*\{a, bb\} = \{a, bb, aa, abb, ba, bbb, \dots\}$

所以 $\mathbf{L}((\boldsymbol{a} + \boldsymbol{b})^*(\boldsymbol{a} + \boldsymbol{bb})) = \{$ $\boldsymbol{w}|\boldsymbol{w}$由$a$和$b$组成，仅以$a$或$bb$结尾$\}$。

主要内容

- 正则表达式
- **有穷自动机和正则表达式**
  - 由自动机到正则表达式
  - 由正则表达式到自动机

- **DFA，NFA，ε-NFA和正则表达式在表示语言的能力上是等价的**
  - 相互之间的转换关系



DFA到正则表达式

- **定理：若 $L = \mathbf{L}(A)$ 是某 DFA A 的语言，那么存在正则表达式R满足$L = \boldsymbol{L}(R)$。**
- **证明方法：**
  - 递归法
  - 状态消除法

DFA->正则表达式:递归法

- 对A的状态进行编号，令1为开始状态，即

$$A = (\{1,2,3,\ldots,n\}, \Sigma, \delta, 1, F)$$

设正则表达式 $R_{i,j}^{(k)}$ 表示从 i 到 j 但中间节点状态编号不超过 k 全部路径的字符串集（简称k-路径）：

$$R_{i,j}^{(k)} = \{x \mid \hat{\delta}(i,x) = j, x\text{经过的状态除两端外都不超过 } k\}$$



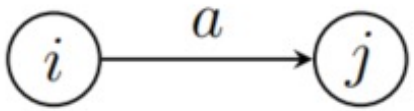
注意，路径的起点和终点都不是“中间的”

DFA->正则表达式:递归法

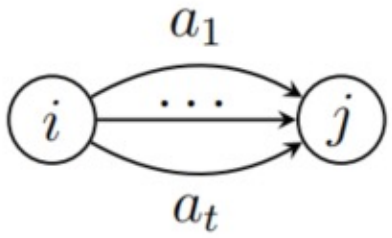
- **归纳基础:$k=0$, $R_{i,j}^{(0)}$为从 i 直接到 j ，没有经过任何中间节点的字符串集合**
- **情况1：i不等于j**
  - 没有i到j的状态转移 $R_{i,j}^{(0)}=\emptyset$



  - 有一个i到j的转移 $R_{i,j}^{(0)}=\boldsymbol{a}$



  - 有多个i到j的转移 $R_{i,j}^{(0)}=\boldsymbol{a_1}+\cdots+\boldsymbol{a_t}$



DFA->正则表达式:递归法

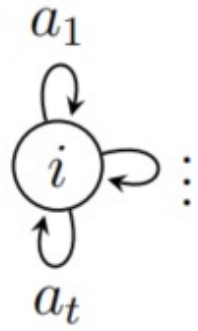
- **归纳基础:$k=0$, $R_{i,j}^{(0)}$为从 i 直接到 j ，没有经过任何中间节点的字符串集合**
- **情况2: i等于j**
  - 状态i没有到自身的转移$R_{i,j}^{(0)}=\boldsymbol{\varepsilon}$



  - 状态i有一个到自身的转移$R_{i,j}^{(0)}=\boldsymbol{a}+\boldsymbol{\varepsilon}$



  - 状态i有多个到自身的转移$R_{i,j}^{(0)}=\boldsymbol{a_1}+\cdots+\boldsymbol{a_t}+\boldsymbol{\varepsilon}$



k —路径的情况

- $R_{i,j}^{(k)}$ **有两种情况：**
  - 不经过状态k
  - 经过状态k一次或者多次
- $R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} \left(R_{k,k}^{(k-1)}\right)^* R_{k,j}^{(k-1)}$

不经过k的路径集合

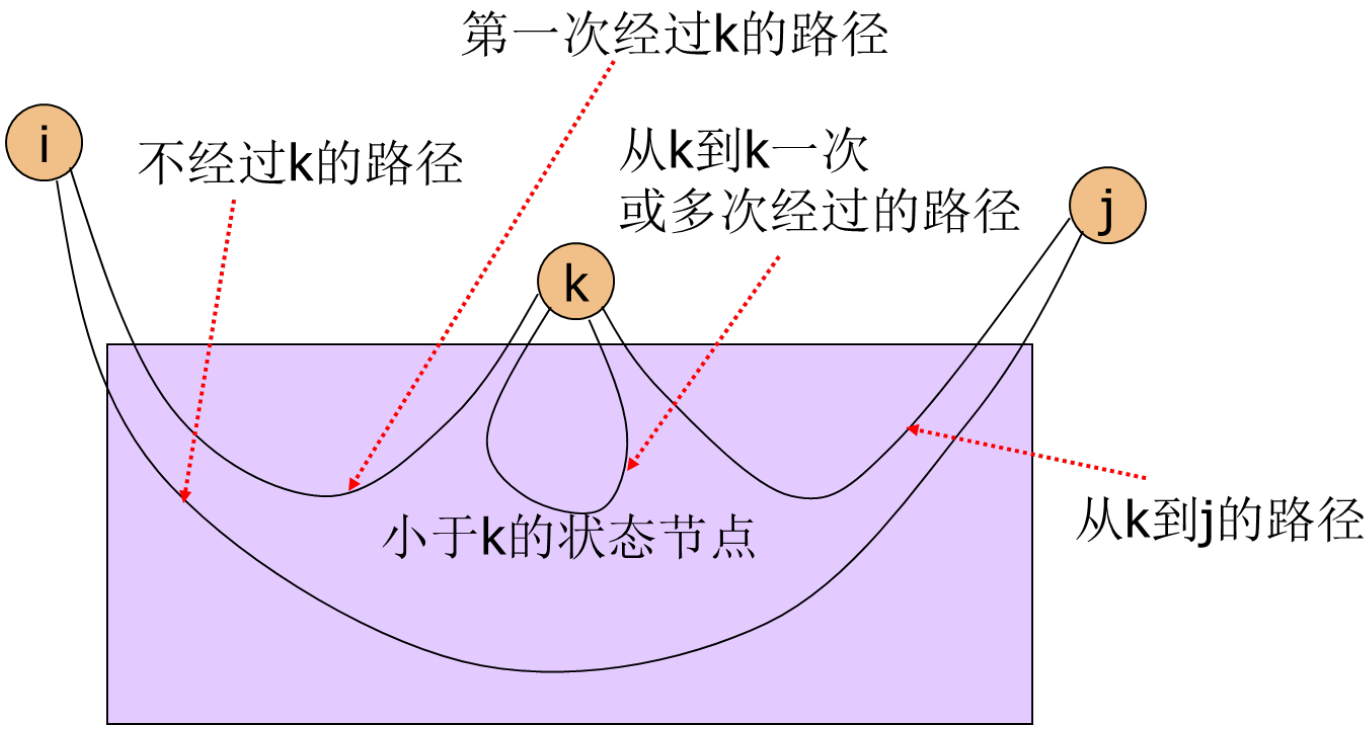
首次从i到k

一次或多次从k到k

最后，从k到j

递归关系示意图

- **递归关系：** $R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} \ (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$



- **运用归纳假设可以证明$R_{i,j}^{(k)}$可以用正则表达式表示！**

DFA->正则表达式

- 如果1是开始结点，则和**DFA等价的正则表达式**就是

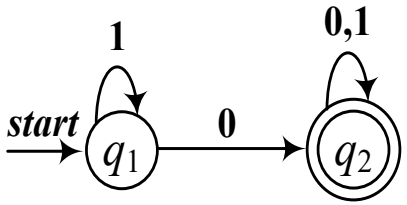
$$\bigcup_{j \in F} R_{1,j}^{(n)}$$

即，从状态1到终结状态的所有路径的集合。

□

DFA->正则表达式示例

- **把下图所示DFA转换为正则表达式。这个DFA接受至少含有一个0 的串。**



$k=0$的情况(状态i直接到状态j):

$R_{1,1}^{(0)}$	$\varepsilon+1$
$R_{1,2}^{(0)}$	0
$R_{2,1}^{(0)}$	$\emptyset$
$R_{2,2}^{(0)}$	$\varepsilon+0+1$

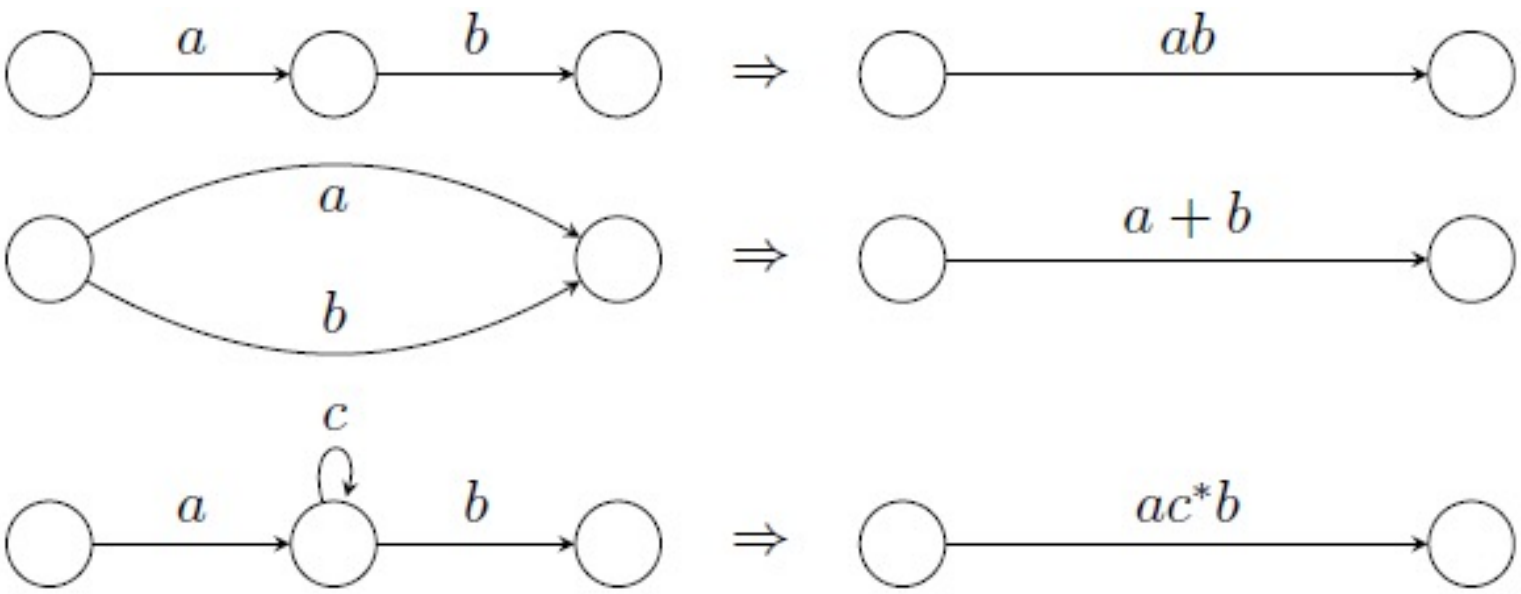
$R_{1,1}^{(1)}$	$\varepsilon+1+(\varepsilon+1)(\varepsilon+1)^*(\varepsilon+1)$
$\boldsymbol{R_{1,2}^{(1)}}$	$0+(\varepsilon+1)(\varepsilon+1)^*0$
$R_{2,1}^{(1)}$	$\emptyset+\emptyset(\varepsilon+1)^*(\varepsilon+1)$
$\boldsymbol{R_{2,2}^{(1)}}$	$\varepsilon+0+1+\emptyset(\varepsilon+1)^*0$

$R_{1,1}^{(2)}$	$1^*+1^*0(\varepsilon+0+1)^*\emptyset$
$\boldsymbol{R_{1,2}^{(2)}}$	$1^*0+1^*0(\varepsilon+0+1)^*(\varepsilon+0+1)$
$R_{2,1}^{(2)}$	$\emptyset+(\varepsilon+0+1)\ (\varepsilon+0+1)^*\emptyset$
$R_{2,2}^{(2)}$	$\varepsilon+0+1+(\varepsilon+0+1)(\varepsilon+0+1)^*(\varepsilon+0+1)$

DFA->正则表达式:状态消除法

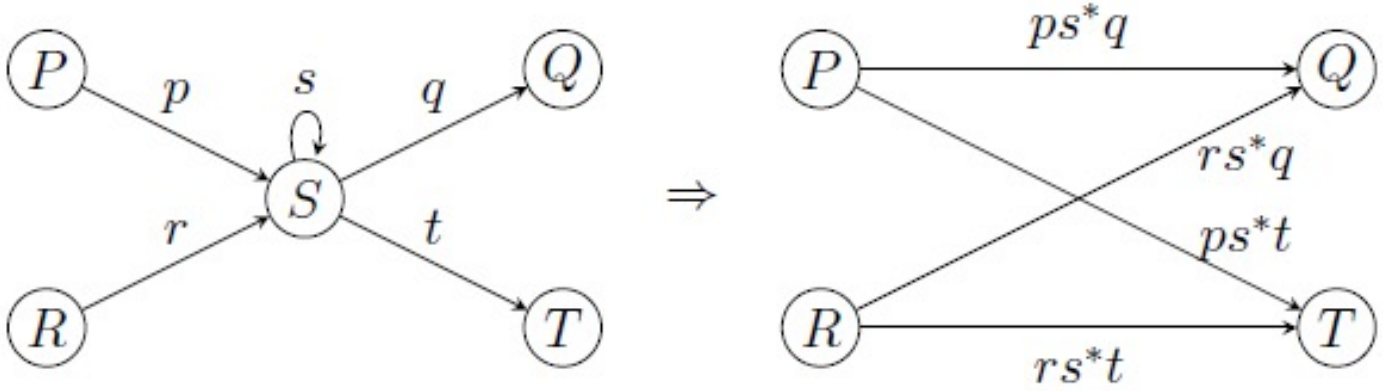
- **从DFA中逐个删除状态**
  - 会消除掉状态的原有路径，因此要补上等价的路径
- **用标记了正则表达式的新路径替换被删掉的路径**
- **保持“自动机”等价**

- 正则表达式运算（连接，加，闭包）和DFA中路径的关系



状态消除法

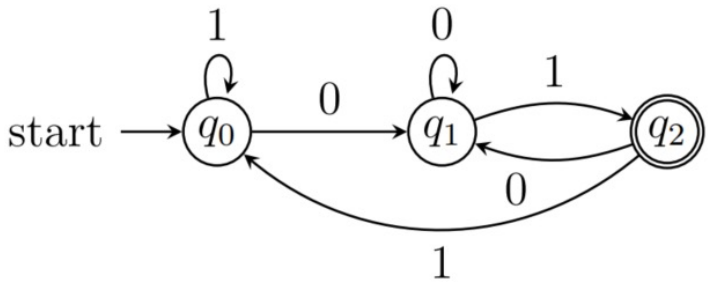
- 更一般的情况，要为被删除的状态S的每个“入”和“出”路径的组合，补一条等价的新路径，并用新的正则表达式表示



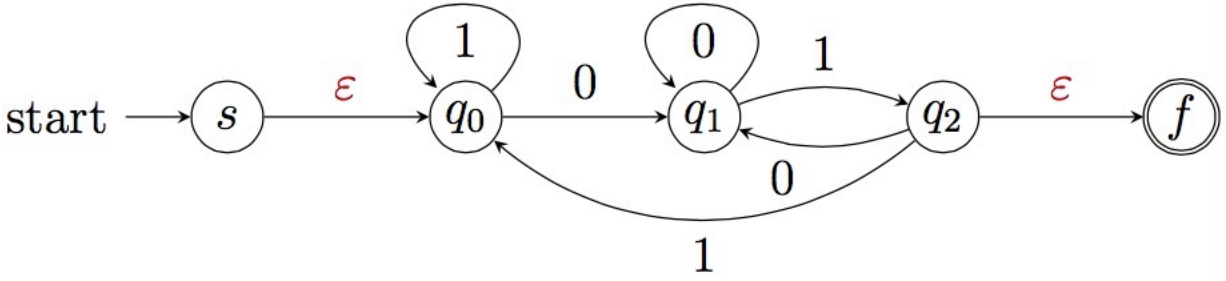
- 2条入，2条出，要补4条（P到Q，P到T，R到Q，R到T）

状态消除法示例

- **例:由0和1构成的串中，接受全部以01结尾的串。**
- DFA:

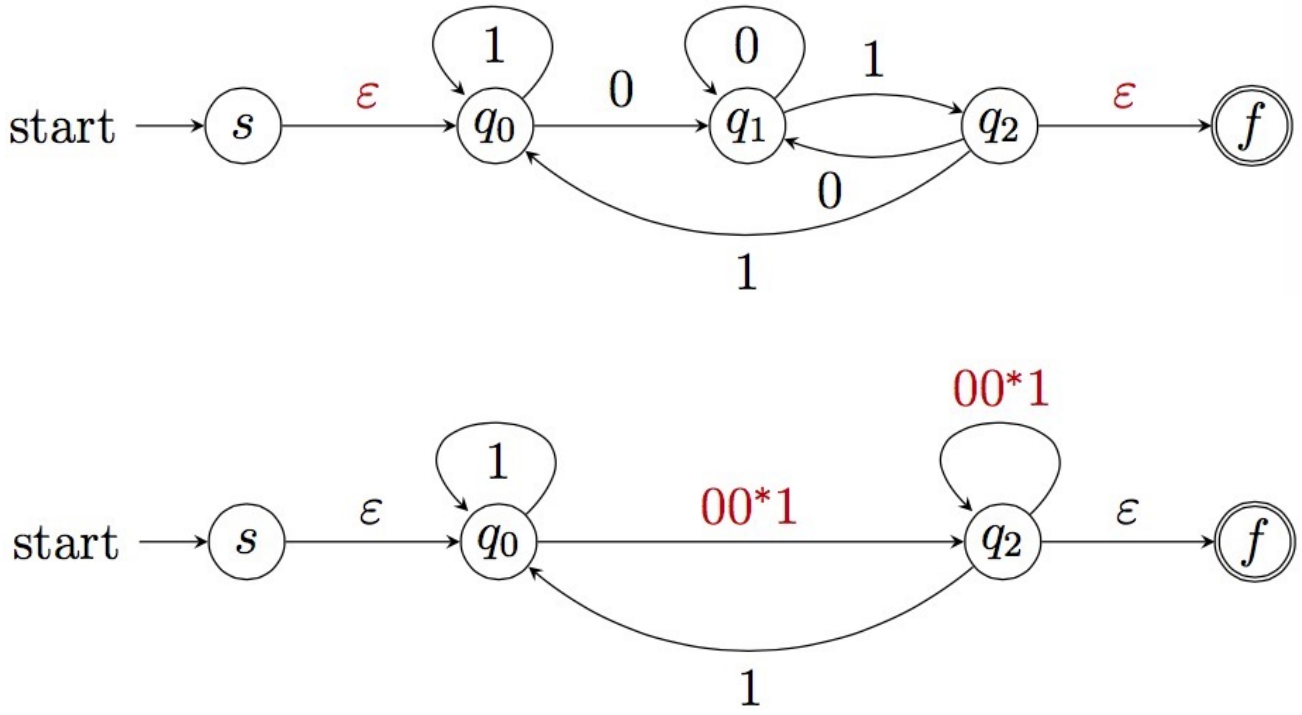


(1)利用空转移，增加新的开始(*s*)和结束状态(*f*)



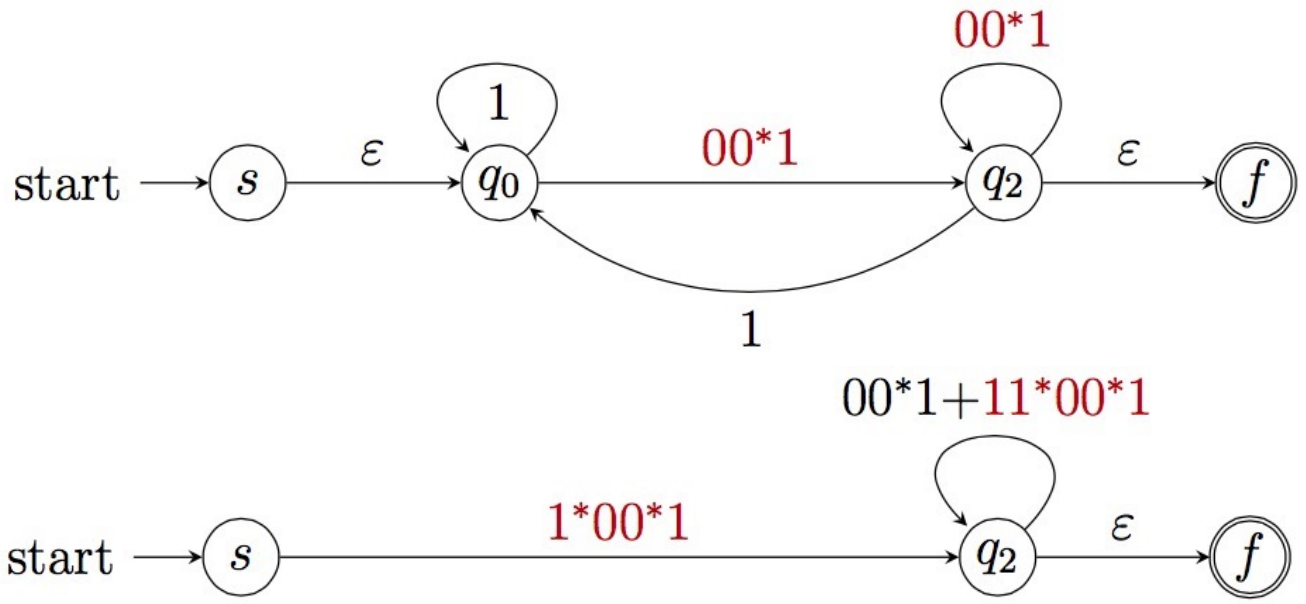
状态消除法示例

(2) 消除状态q_1，需要增加其“入”度$\{q_0, q_2\}$和“出”度$\{q_2\}$之间的边$q_0 00^* 1 q_2$和$q_2 00^* 1 q_2$，即$q_0 \to q_2$和$q_2 \to q_2$



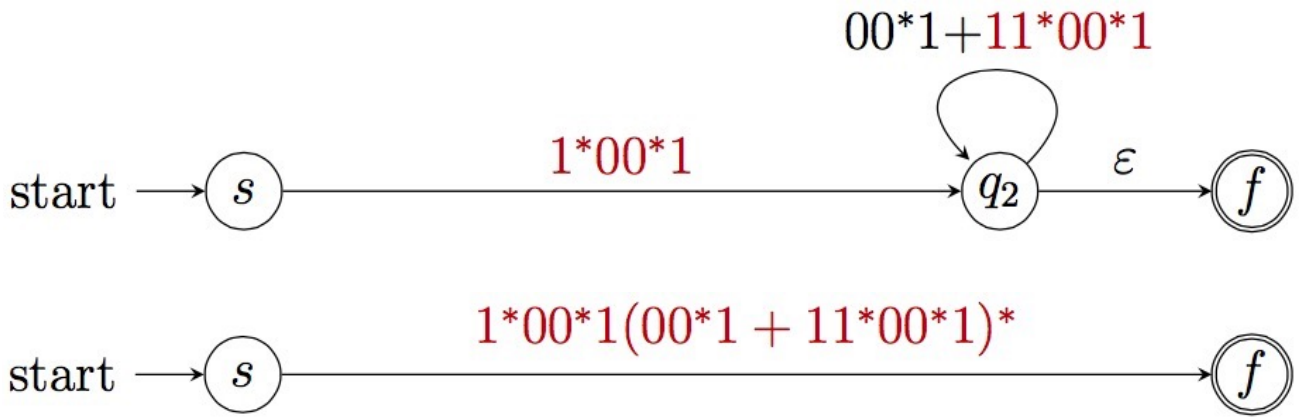
状态消除法示例

(3)消除状态q_0，需要增加其“入”度$\{s, q_2\}$和“出”度$\{q_2\}$之间的边$s1^*00^*1q_2$和$q_2 11^*00^*1q_2$，即$s \to q_2$和$q_2 \to q_2$



状态消除法示例

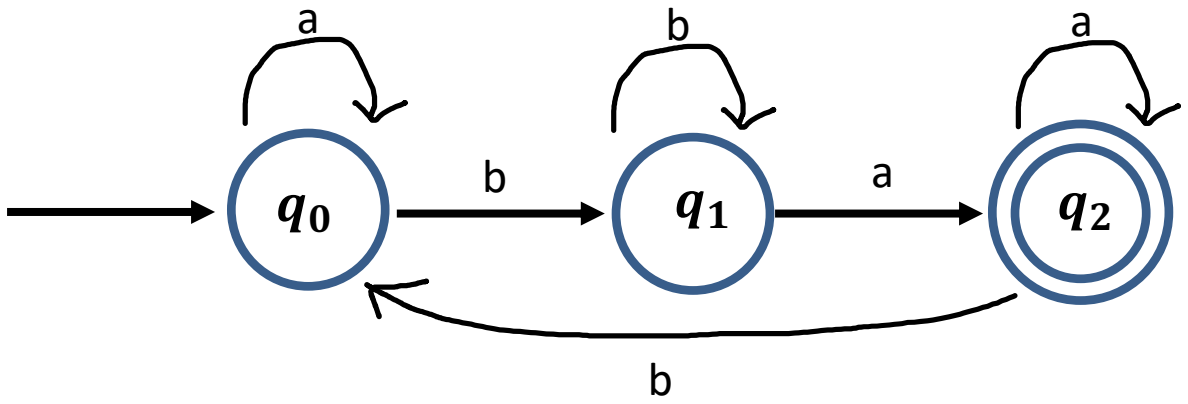
（4）消除状态q_2，需要增加其“入”度$\{s\}$和“出”度$\{f\}$之间的边$s1^*00^*1(00^*1+11^*00^*1)^*f$，即$s \to f$



因此，该自动机的正则表达式式为：

$$1^*00^*1(00^*1+11^*00^*1)^*$$

- 试求解下面DFA所对应的正则表达式



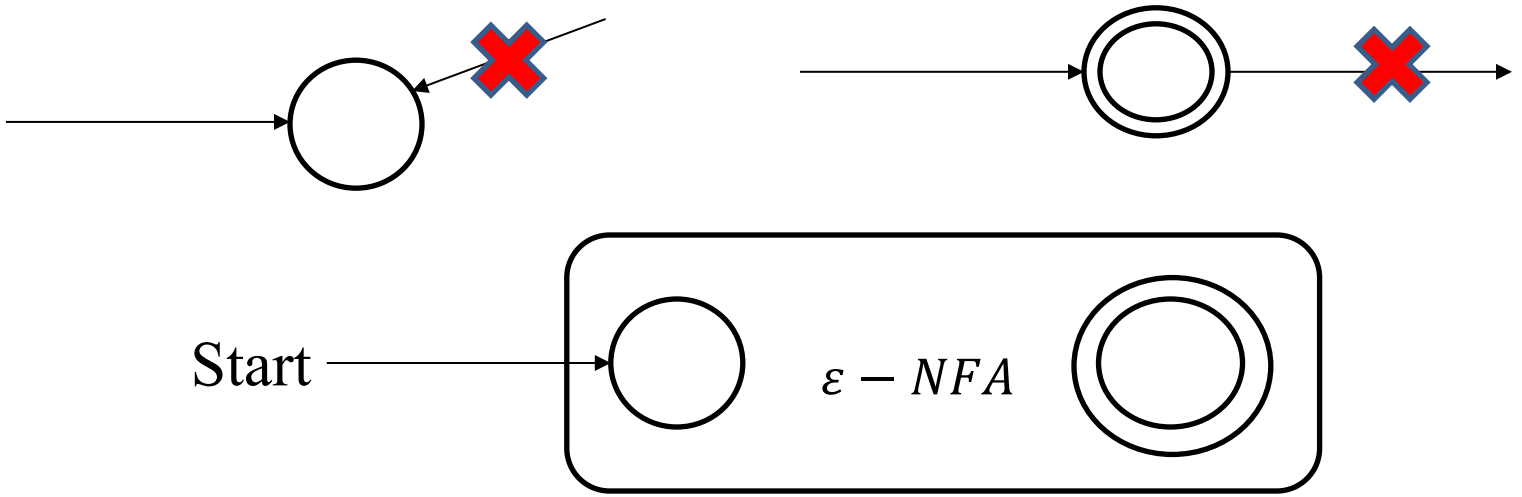
- 正则表达式
  - 正则表达式的递归定义
  - 正则表达式的示例
- **有穷自动机和正则表达式**
  - 由自动机到正则表达式
  - 由正则表达式到自动机

正则表达式到自动机

- **定理：每个正则表达式定义的语言，都可被有穷自动机识别。**

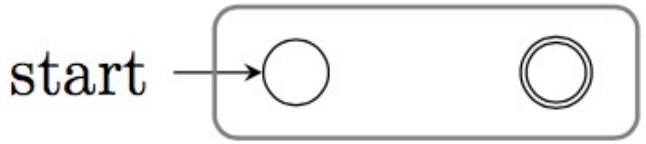
⇔任何正则表达式R，都存在与其等价的ε-NFA A，即$\mathbf{L}(A) = \mathbf{L}(R)$，并且$A$满足：

- 仅有一个接受状态；
- 没有进入开始状态的边；
- 没有离开接受状态的边。

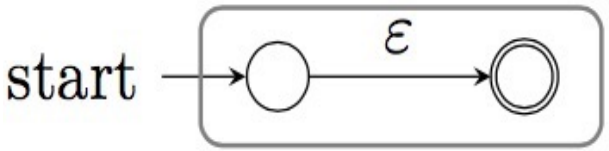


- 归纳基础：

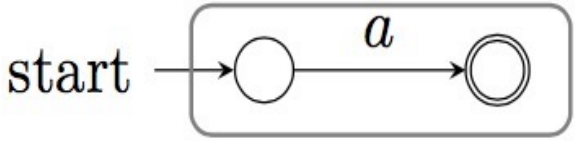
1. 对正则表达式$\emptyset$，有ε−NFA



2. 对正则表达式ε，有ε−NFA



3. $\forall a \in \Sigma$，对正则表达式a，有ε−NFA



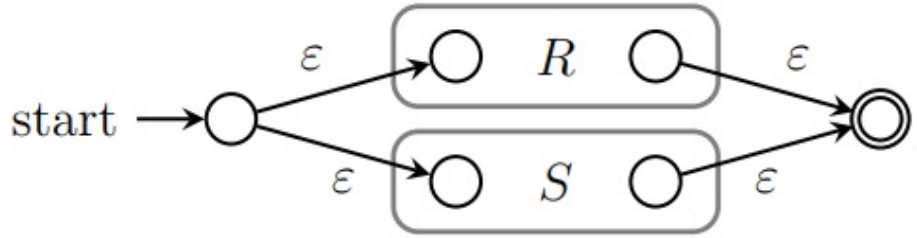
- 归纳递推：

假设若r和s为正则表达式，则它们对应的ε-NFA分别为R和S（归纳假设：在子正则表达式上成立）。



- **归纳递推：**

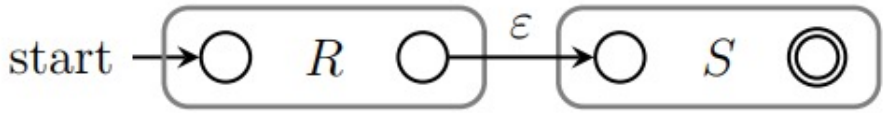
$r+s$的ε−NFA:



- 添加一个开始状态和一个接受状态；
- 将R和S通过空转移以并的方式连接起来。

- **归纳递推：**

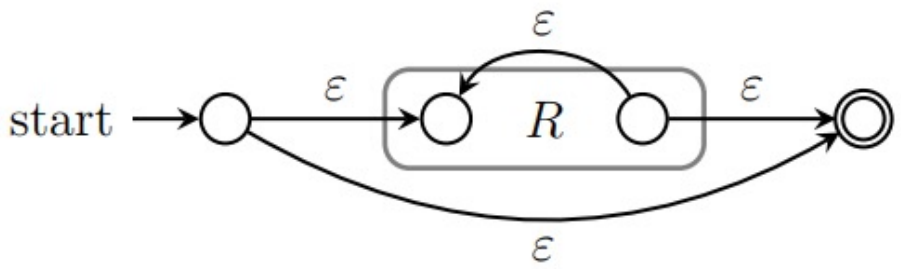
rs的ε−NFA:



- 通过空转移将 R 的接受状态和 S 的开始状态连接起来。

- **归纳递推：**

r^*的ε−NFA:

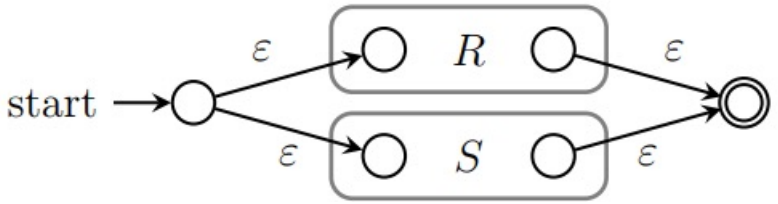


- 添加一个开始状态和一个接受状态；
- 通过空转移将开始状态连接，将接受状态连接；
- 把原来的接受状态改为非接受状态，通过空转移跳转回原来的开始状态（表示闭包里的 ≥0次）；
- 闭包里的0次通过空转移连接开始状态和接受状态实现。

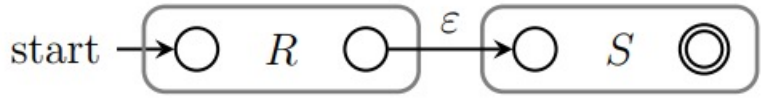
- **归纳递推：**

若r和s为正则表达式，则它们对应的ε−NFA别为R和S。

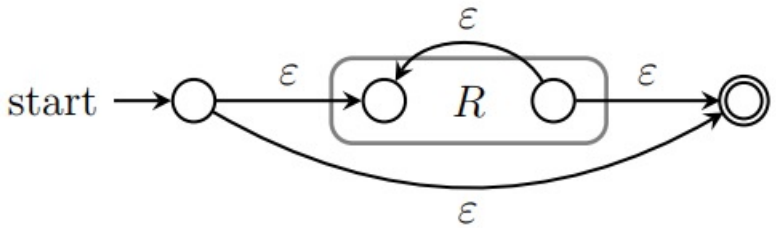
$r+s$的ε−NFA:



rs的ε−NFA:



r^*的ε−NFA:

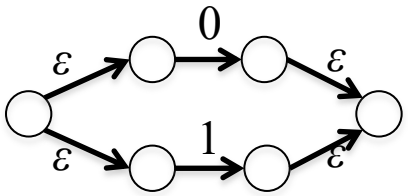


因此对于任何正则表达式，可以构造等价的ε-NFA 。

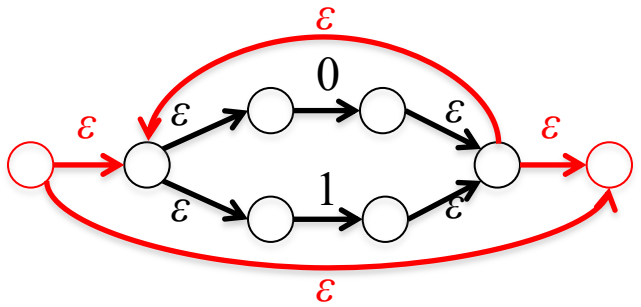
示例

- **例：正则表达式** $(0+1)^*1(0+1)$ **构造为ε-NFA。**

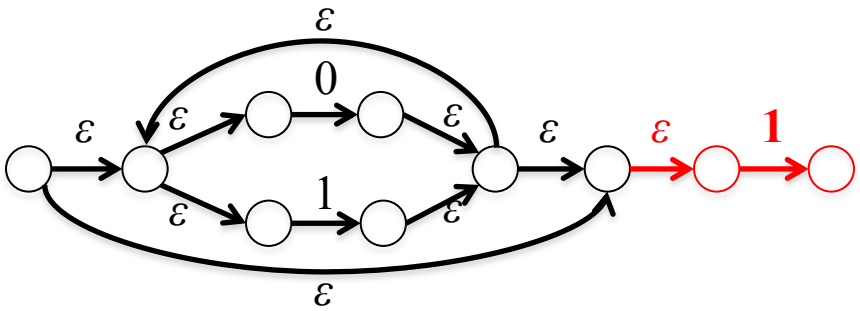
（1）构造(0+1)



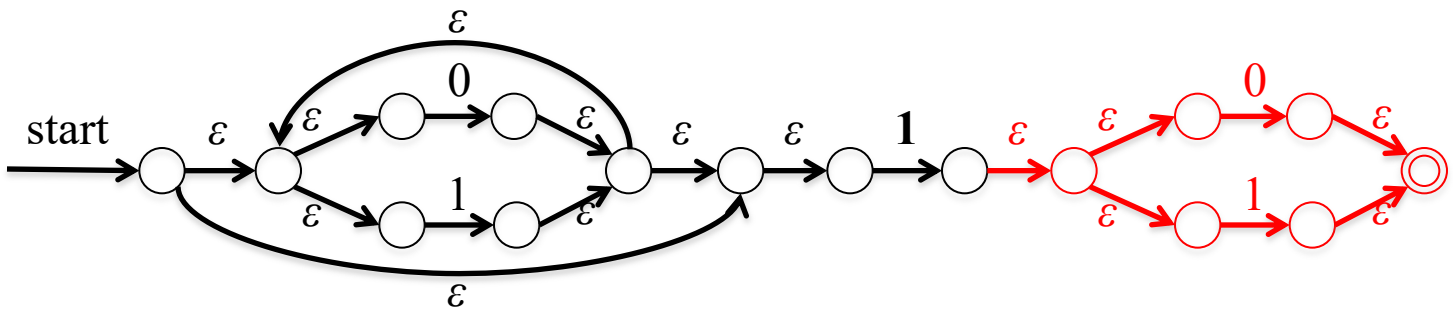
（2）构造$(0+1)^*$



（3）构造$(0+1)^*1$

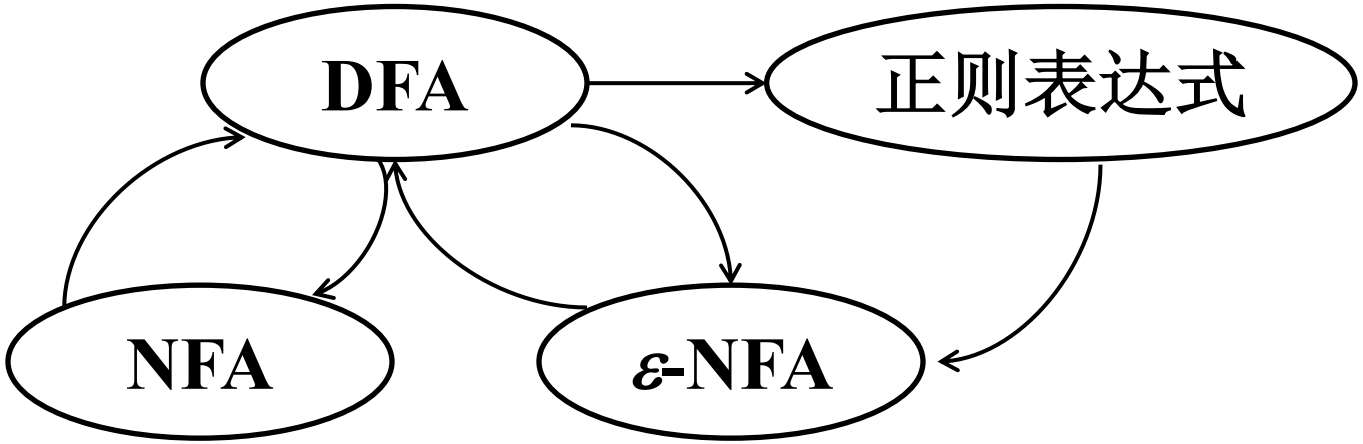


（4）构造(0+1)*1(0+1)



小结

- **正则表达式基本概念**
- **正则表达式与有穷自动机的转换**
  - 正则表达式→DFA
  - DFA→正则表达式：构造法＋状态消除法
- **正则表达式的代数定律**



正则表达式

例1：设计正则表达式 E 使得

$$L(E) = \{w | w\text{长度为2且第2个字符是1}\}$$

主观题 10分

设置

例2：设计正则表达式 E 使得

$$L(E) = \{w | w\text{至少包含3个连续的0}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

主观题 10分

设置

例3：设计正则表达式 E 使得

$$L(E) = \{w | w\text{以}01\text{结尾}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

例4：设计正则表达式 E 使得

$$L(E) = \{w | w\text{以1开头且以0结尾}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

主观题 10分

设置

例5：设计正则表达式 E 使得

$$L(E) = \{w | w\text{的开头字符和结尾字符相同}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

例6：设计正则表达式 E 使得

$$L(E) = \{w | w\text{由交替的0和1组成}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

主观题 10分

设置

例7：设计正则表达式 E 使得

$$L(E) = \{w | w\text{从右端数第5个位置是1}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

例8：设计正则表达式 E 使得

$$L(E) = \{w | w\text{的前5位至少包含一个}1 \wedge |w| \geq 1\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

主观题 10分

设置

例9：设计正则表达式 E 使得

$$L(E) = \{w | w\text{的后5位至少包含一个}1 \wedge |w| \geq 1\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

主观题 10分

设置

例10：设计正则表达式 E 使得

$$L(E)=\{w|w\text{中至少包含一个}a\text{和一个}b\}$$

该语言的字母表包含 a、b和c。

正常使用主观题需2.0以上版本雨课堂

作答

主观题 10分

设置

例11：设计正则表达式 E 使得

$L(E) = \{w | w$中每对相邻的0都出现在每对相邻的1之前$\}$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

主观题 10分

设置

例13：设计正则表达式 E 使得

$$L(E)=\{w|w\text{中既不包含子串}00,\ \text{也不包含子串}11\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

主观题 10分

设置

例14：设计正则表达式 E 使得

$$L(E) = \{0^m 1^n | m \geq 0 \wedge n \geq 0 \wedge m+n\text{是偶数}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

主观题 10分

设置

例15：设计正则表达式 E 使得

$$L(E) = \{w | \text{当} w \text{以0结尾时，其长度是奇数} \wedge |w| \geq 1\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

主观题 10分

设置

例16：设计正则表达式 E 使得

$$L(E) = \{w | w\text{的前5位至少有一个子串}00 \wedge |w| \geq 2\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

主观题 10分

设置

例17：设计正则表达式 E 使得

$$L(E) = \{w | w\text{的第2位至第5位至少有一个}1 \wedge |w| \geq 2\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

主观题 10分

设置

例18：设计正则表达式 E 使得

$$L(E)=\{w|w\text{至少包含3个1，且倒数第3位为1}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答